

DSA FROM SCRATCH

# to Advanced

DAY 09 / LINEAR SEARCH

Concept → Example → Implementation → Complexity

Pattern → Problems → Takeaway

GITHUB RESOURCES

[https://github.com/ritesh2687/dsa\\_ppt\\_post](https://github.com/ritesh2687/dsa_ppt_post)

A structured learning series for beginners, students & interview preparation

# Today's Learning Roadmap

02

A guided path from the first idea to interview revision.

**01** What is Linear Search?

**02** Why do we need it?

**03** How does it work?

**04** Example + Dry Run

**05** C++ Implementation

**06** Time & Space Complexity

**07** Pattern + Variations

**08** Edge Cases + Mistakes

**09** LeetCode Problems

**10** Interview Tip

**11** Key Takeaway

**12** Tomorrow: Binary Search

# What is Linear Search?

Sequentially check each element until the answer is known.

## THE IDEA

**Linear Search checks elements one by one.**

- Start from index 0
- Compare the current element with the target
- If equal → return the index
- Otherwise move to the next element
- If the collection ends → return -1

## MENTAL MODEL

**START**



**COMPARE**



**MOVE**



**STOP**

Also called: Sequential Search

# Why Do We Need Linear Search?

Simple is valuable when the data or constraints are simple.

## ADVANTAGES

- Works on unsorted data
- Very simple to implement
- Easy to understand and debug
- Useful for small datasets
- No preprocessing required
- Good when only one/few searches are needed

## LIMITATION

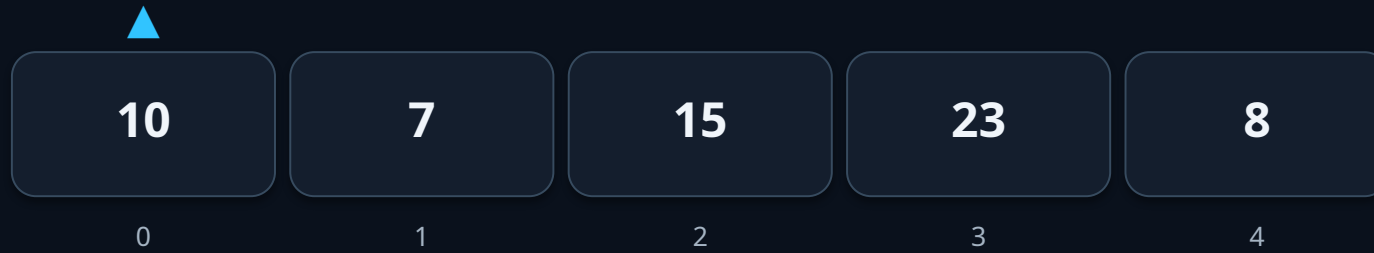
**$O(n)$**

**Worst-case time**

If the target is last—or absent—we may inspect every element.

# How Linear Search Works

Example: find 23 in [10, 7, 15, 23, 8].



Start at index 0

Compare

Move right

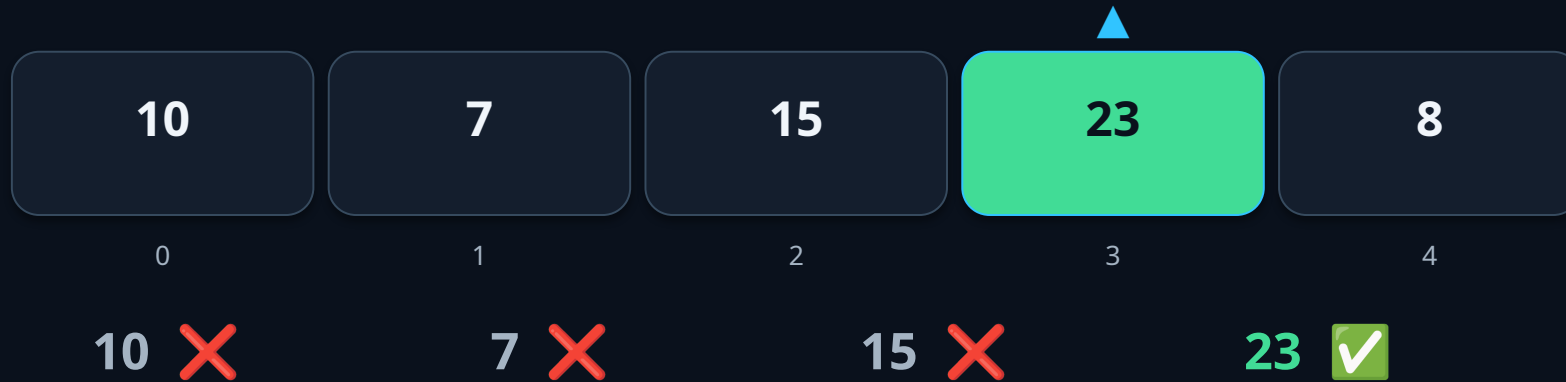
Stop when equal

**Target = 23**

If no match exists, return -1.

# Visual Example

The pointer moves left → right until the target is found.



**Answer = index 3**

Remember: arrays use zero-based indexing → first element is index 0.

# Dry Run

Track the index, value, comparison, and action.

| i | arr[i] | Comparison   | Action   |
|---|--------|--------------|----------|
| 0 | 10     | 10 $\neq$ 23 | Continue |
| 1 | 7      | 7 $\neq$ 23  | Continue |
| 2 | 15     | 15 $\neq$ 23 | Continue |
| 3 | 23     | 23 = 23      | Return 3 |

The moment the condition is satisfied, return immediately.

# C++ Implementation

A minimal function: scan, compare, return index.

C++

```
int linearSearch(int arr[], int n, int target) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == target)  
            return i;  
    }  
    return -1;  
}
```

READ THE CODE

- arr[] → input array
- n → number of elements
- target → value to find
- for loop → visits every index
- return i → found at index i
- return -1 → target not found

# Complete C++ Example

Runnable program + expected output.

**C++**

```
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}

int main() {
    int arr[] = {10, 7, 15, 23, 8};
    int n = 5, target = 23;
    int result = linearSearch(arr, n, target);
    cout << "Index: " << result;
    return 0;
}
```

**OUTPUT**

**Index: 3**

**KEY IDEA**

**Found → return immediately.**  
**Absent → return -1.**

# Modern C++: vector Version

Use `const vector<int>&` to avoid copying the array.

C++

```
int linearSearch(const vector<int>& nums, int target) {  
    for (int i = 0; i < nums.size(); i++) {  
        if (nums[i] == target)  
            return i;  
    }  
    return -1;  
}
```

WHY `const &` ?

- vector is a flexible C++ container
- `const` → function does not modify it
- `&` → avoids copying the whole vector
- Result: clean and efficient parameter passing

Beginner rule: understand the loop first; learn the parameter syntax step-by-step.

# Best Case

Target is the first element.



Only 1 comparison

TIME COMPLEXITY

$O(1)$

The target is found immediately.

# Average Case

The target is typically found somewhere in the search.



AVERAGE TIME

$O(n)$

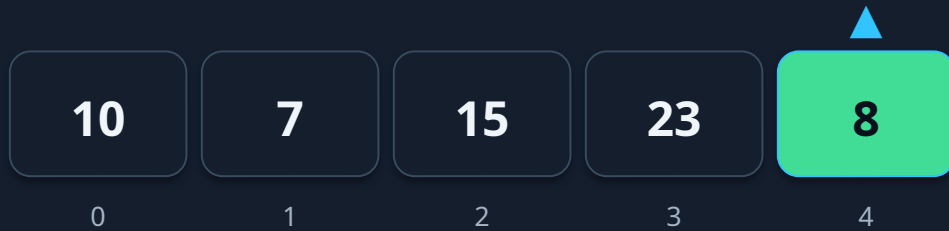
Big-O describes growth; it is not a claim that the algorithm always performs exactly  $n/2$  operations.

# Worst Case

13

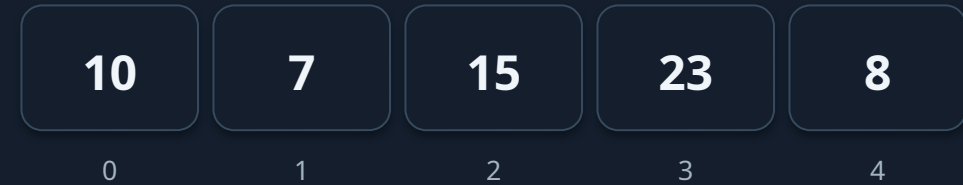
Two common ways to inspect every element.

## CASE 1 — LAST ELEMENT



Target = 8

## CASE 2 — ABSENT



Target = 99

**All elements are checked →  $O(n)$**

Extra space =  $O(1)$

# Complexity Summary

Interview-ready complexity at a glance.

BEST

$O(1)$

AVERAGE

$O(n)$

WORST

$O(n)$

SPACE

$O(1)$

Only a few variables; no extra array is created.

# Linear Search vs Binary Search

Choose based on the data and the problem constraints.

| FEATURE          | LINEAR SEARCH    | BINARY SEARCH    |
|------------------|------------------|------------------|
| Data requirement | Unsorted allowed | Sorted required  |
| Approach         | Sequential       | Divide & Conquer |
| Best             | $O(1)$           | $O(1)$           |
| Worst            | $O(n)$           | $O(\log n)$      |
| Simplicity       | Very easy        | Moderate         |
| Best use         | Small / unsorted | Large / sorted   |

**Interview statement: Binary Search is not automatically better — its sorted-data requirement matters.**

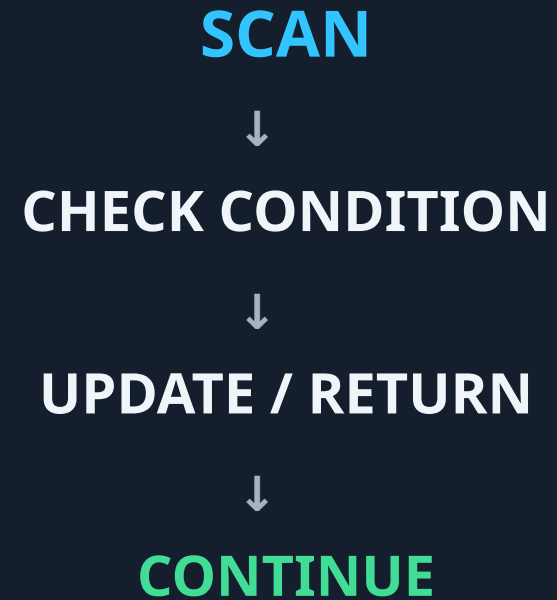
# Recognizing the Linear Search Pattern

When the task says “scan and decide,” think linear scan.

## COMMON USE CASES

- Find an element
- Find first occurrence
- Check whether an element exists
- Count matching elements
- Find minimum / maximum
- Scan for a condition
- Find the first valid answer

## CORE PATTERN



# Variations of Linear Search

One scan pattern can solve many small problems.

## First occurrence

```
if (arr[i] == target) return i;
```

## Check existence

```
if (arr[i] == target) return true;
```

## Find maximum

```
maxValue = max(maxValue, arr[i]);
```

## Count occurrences

```
if (arr[i] == target) count++;
```

Same engine: visit each element once, then apply the condition you need.

# Edge Cases

18

Always test the boundaries before trusting your solution.

**Empty array**

**Single element**

**Target first**

**Target last**

**Target absent**

**Duplicate values**

**Multiple occurrences**

Expected behavior depends on the requirement: return an index, boolean, count, or another value.

# Common Mistakes

Small details can turn a correct idea into a wrong submission.

- 1 Off-by-one errors
- 2 Returning the value instead of the index
- 3 Forgetting return -1
- 4 Continuing after the required first match
- 5 Ignoring duplicate-value requirements
- 6 **Saying Linear Search is always  $O(n)$  — remember best case  $O(1)$**

# LeetCode #1 — Two Sum

Connect brute-force scanning with a classic problem.

## THE IDEA

- Find two indices whose values add to the target
- A brute-force solution checks pairs systematically
- This is closely related to repeated scanning
- Brute force is  $O(n^2)$
- Hashing improves the approach

## DSA LESSON

**Start simple → understand the scan → optimize when constraints demand it.**

Don't memorize the optimized code first.  
Understand why brute force works and why it is slow.

# LeetCode #724 — Find Pivot Index

A scanning problem where prefix/suffix information helps.

## WHAT IT ASKS

- Find an index where left-side sum equals right-side sum
- The array can be scanned from left to right
- Track the information needed for the next decision
- The key pattern is systematic scanning

## PATTERN

### SCAN

→ **maintain totals** → **compare** → **return**

A good reminder: a linear scan can do more than “find a value.”

# LeetCode #35 — Search Insert Position

A natural bridge from Linear Search to Binary Search.

## SEARCH CONCEPT

- Input is sorted
- Find the target's index or insertion position
- A simple scan can reason about the answer
- But the sorted property gives us more power

## NEXT STEP

**SORTED DATA**



**ELIMINATE HALF**



**BINARY SEARCH**

Expected optimized approach:  $O(\log n)$

# Interview Tip

Before coding, ask what exactly needs to be returned.

## CLARIFY THE OUTPUT

- Any occurrence?
- First occurrence?
- Last occurrence?
- All occurrences?
- Boolean existence?
- Value or index?

## ONE MORE QUESTION

**Is the data sorted?**

That single detail can change the algorithm you should choose.

# Day 9 Key Takeaway

The interview-ready version to remember.

Sequential scan

Checks elements one by one

Works on unsorted data

No sorting required

Best

$O(1)$

Average

$O(n)$

Worst

$O(n)$

Space

$O(1)$

**Return early when the required answer is found. Understand the requirement before choosing the algorithm.**

# Day 10: Binary Search

25

Today: one-by-one. Tomorrow: eliminate half the search space.

## WHAT'S NEXT?

- Works on sorted data
- Divides the search space
- Much faster for large sorted datasets
- Worst-case complexity:  $O(\log n)$

**"Today we searched one-by-one. Tomorrow, we learn how to eliminate half the search space at every step."**

## GITHUB

### DSA Resources

[https://github.com/ritesh2687/dsa\\_ppt\\_post](https://github.com/ritesh2687/dsa_ppt_post)

**Day 9 complete ✓**